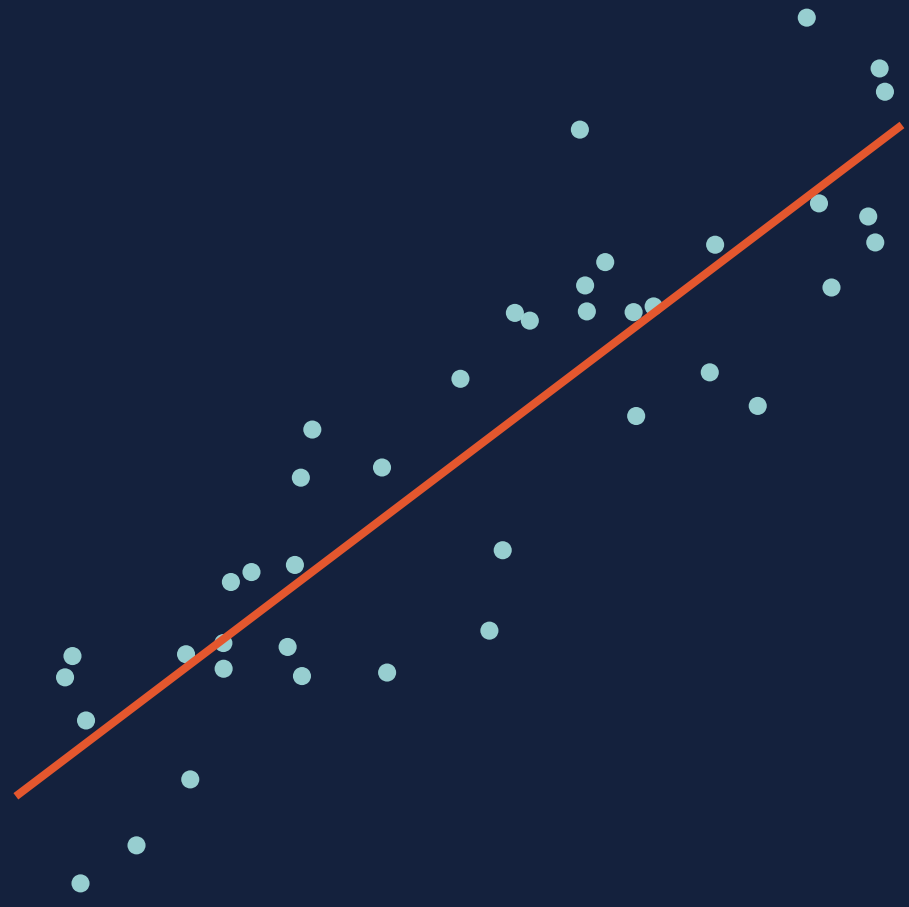
LECTURELOOP DEMO LECTURE

Regression, risk & loss



How a model learns from data: losses, risk, gradient descent, and when a straight line is the wrong tool.

Running example: how long will the harbour ferry take today?

Written by the LectureLoop team · CC BY 4.0

GOALS

What you will be able to do

- Describe the steps that take a question to a trained, tested model
- Write empirical risk for squared loss and explain why we minimise it
- Fit linear and polynomial regression with NumPy and scikit-learn
- Read a cost surface and follow a gradient descent path across it
- Solve multivariate regression in closed form with the normal equation
- Explain why fitting a line to 0/1 labels makes a poor classifier
- Spot under- and overfitting by comparing training and validation error

THE SUPERVISED PIPELINE

From a question to a model in service

1 Frame

Which decision will the model support, and what does a wrong answer cost?

2 Collect

Gather inputs and labels. How were they measured, and by whom?

3 Split

Set aside validation and test data before looking closely.

4 Pick a family

Line, tree, network? Trade capacity against clarity.

5 Loss & metric

A loss to train with, a metric that reflects the decision.

6 Train

Minimise the average loss on the training split.

7 Evaluate

Score on held-out data; hunt for leakage and shift.

8 Ship & watch

Set thresholds, add guardrails, monitor drift.

VOCABULARY

Four ingredients: data, model, loss, metric

Data

Pairs (x, t): a crossing's conditions x and its measured duration t. Stack inputs in X, targets in t.

Model

A function f(x; w) with adjustable weights w. For a line: $f(x) = w_1 x + w_0$.

Loss

ℓ(y, t): the penalty for predicting y when the truth is t. Training minimises it.

Metric

A summary for people deciding if the model is good enough: MAE in minutes, share of crossings within 5 min.

Loss and metric can differ. A metric only needs to be meaningful; a loss also needs to be easy to optimise.

RISK

Expected risk: what we really care about

Picture the harbour as a machine that keeps producing crossings (x, t) from some fixed but unknown distribution D.

- We commit to a model f before the next crossing happens
- Each time it is wrong we pay a loss ℓ(f(x), t)
- What matters is the long-run average on crossings we have not seen yet:

$$R(f) = \mathbb{E}_{(x,\,t)\,\sim\,\mathcal{D}}\,[\,\ell(f(x),\,t)\,]$$

The catch: D is never observed directly. All we have is a finite log of past crossings, so R(f) has to be estimated. That estimate is the next slide.

RISK

Empirical risk: the part we can compute

What we want

$$R(f) = \mathbb{E}_{(x,\,t)\,\sim\,\mathcal{D}}\left[\,\ell(f(x), t)\,\right]$$

Average loss over every crossing that could ever happen. D is unknown.

What we can compute

$$\hat{R}_n(f) = \frac{1}{n}\sum_{i=1}^{n} \ell(f(x^{(i)}), t^{(i)})$$

Average loss over the n crossings we actually logged.

Empirical risk minimisation (ERM)

$$\hat{f} = \arg\min_{f \in \mathcal{F}} \hat{R}_n(f)$$

Pick the model in our class that fits the logged crossings best, and hope it holds up tomorrow.

Other names you will meet for the same quantity: training loss, cost function, objective.

WORKED EXAMPLE

Risk in numbers: four logged crossings

Crossing	True t (min)	Prediction	Squared loss
1	27	30	9
2	31	30	1
3	35	30	25
4	29	30	1

Empirical risk = $(9 + 1 + 25 + 1) / 4 = 9$ min²

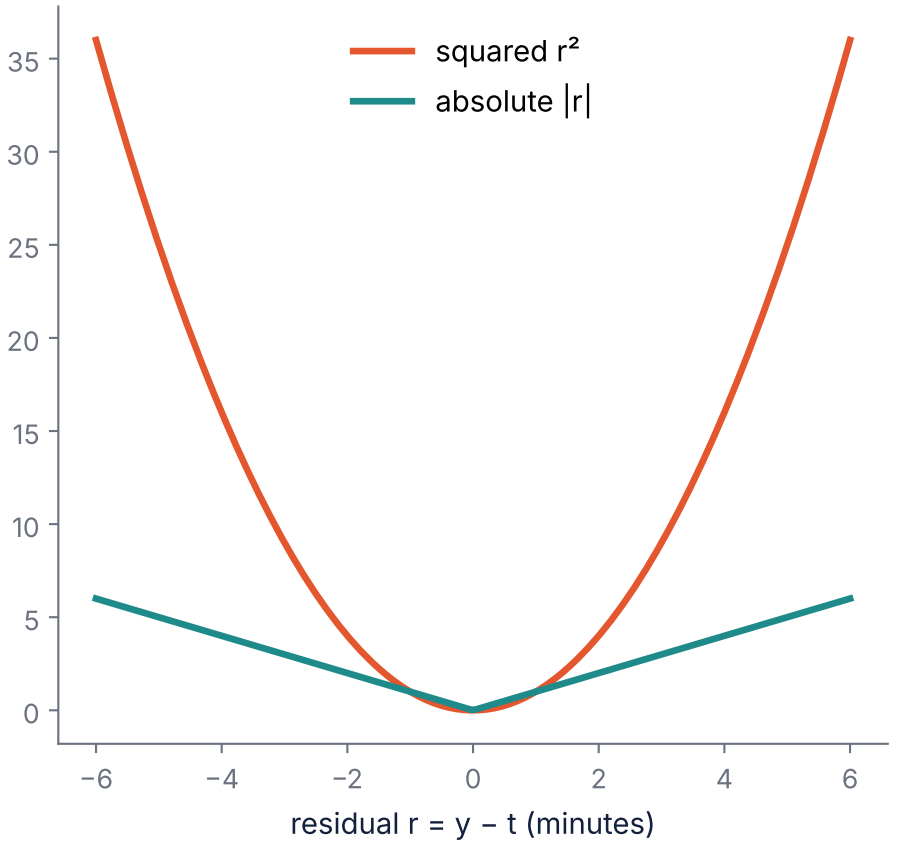
Read it as

A model that always says 30 minutes is off by about 3 minutes on a typical logged crossing (√9).

If tomorrow's crossings look like these, its expected risk should be close to 9 as well. It is still only an estimate.

LOSS FUNCTIONS

Two losses for regression: L2 and L1



Two forecasts for a 30-minute crossing

A says 32 min (off by 2) B says 38 min (off by 8)

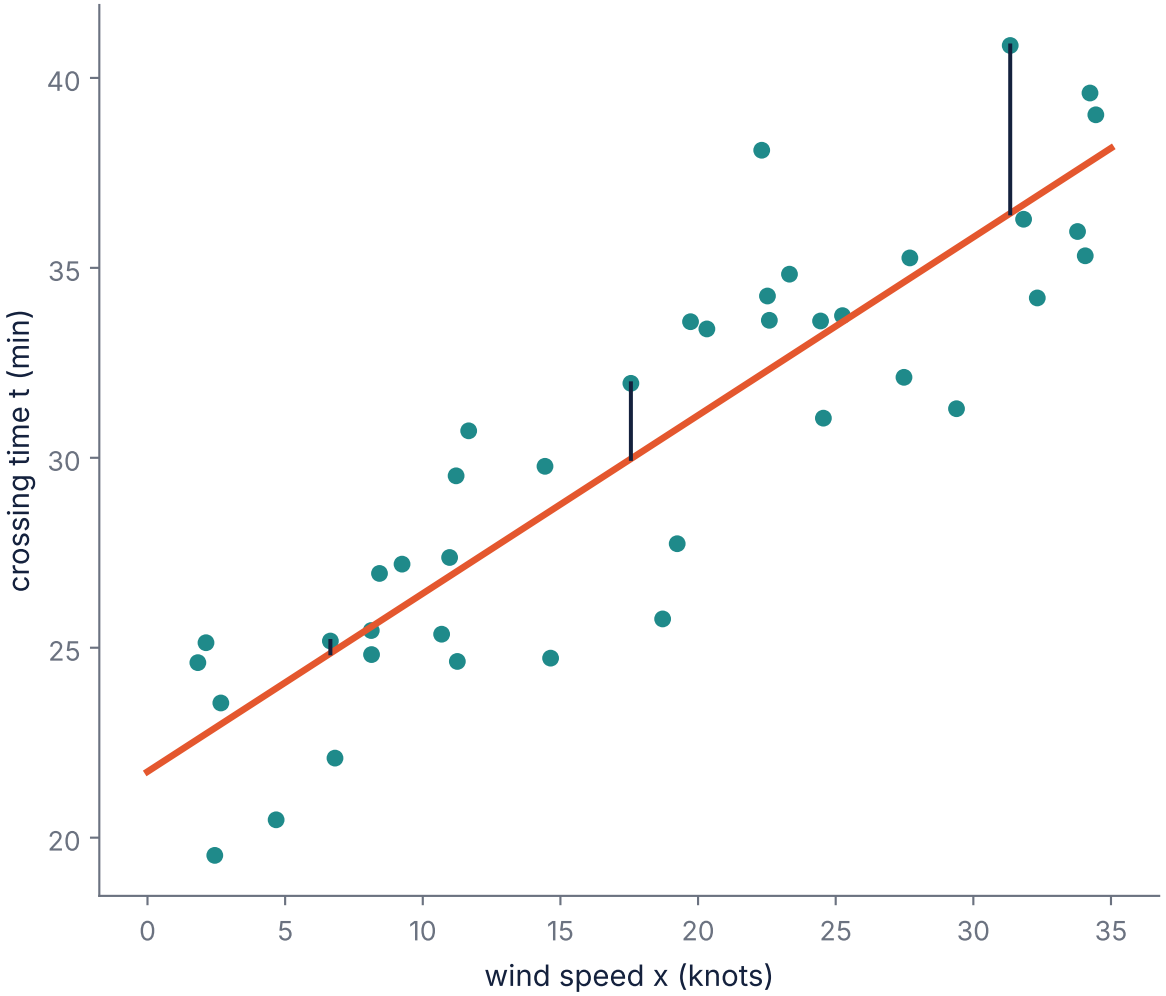
Squared: A pays 4, B pays 64 → B is 16× worse

Absolute: A pays 2, B pays 8 → B is 4× worse

Squared loss (its average is MSE) punishes big misses much harder, so a few storm-day outliers can drag the fit. Absolute loss (MAE) is more forgiving.

1D LINEAR REGRESSION

One input, one line



Model

$$y = w_1 x + w_0$$

w_1: slope, minutes added per knot

w_0: intercept, time in calm water

Residual of example i

$$r^{(i)} = y^{(i)} - t^{(i)}$$

The dark vertical segments. A good line keeps them short overall.

From squared loss to the cost $J(w_1, w_0)$

One crossing: $$\ell(y, t) = \tfrac{1}{2}(y - t)^2$$

The whole log: $$J(w_1, w_0) = \frac{1}{2n}\sum_{i=1}^{n}\left(w_1 x^{(i)} + w_0 - t^{(i)}\right)^2$$

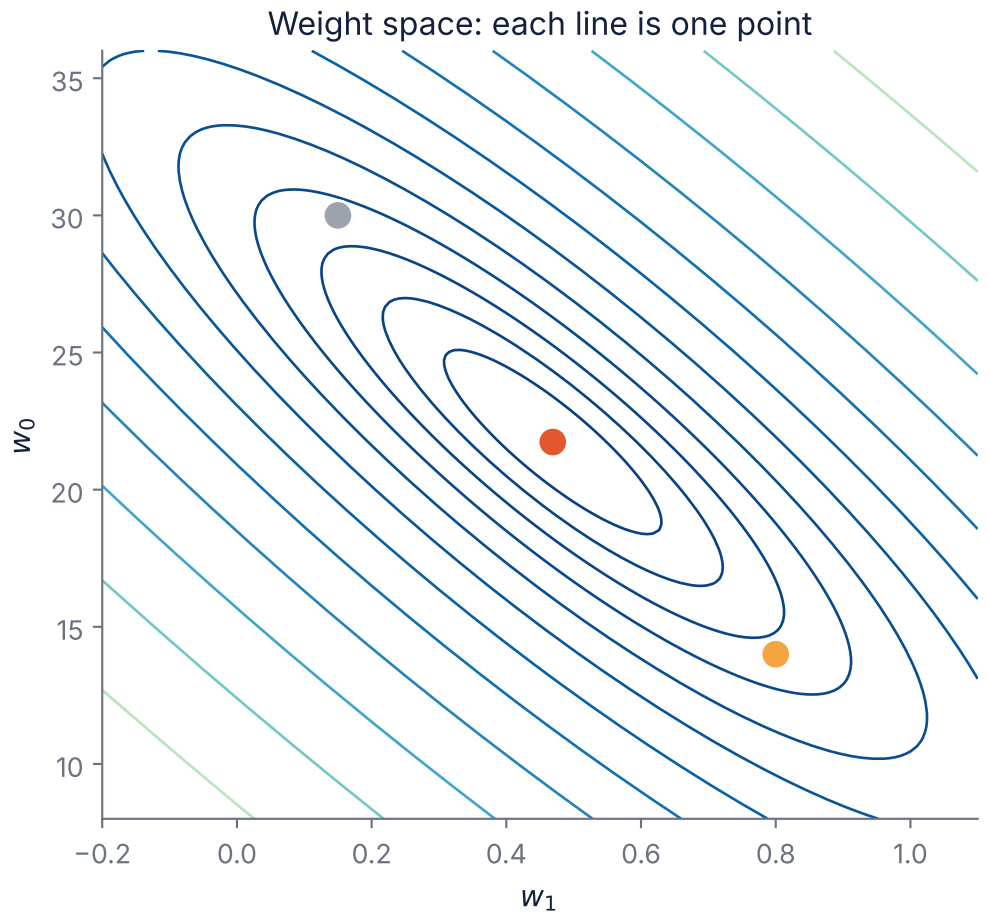
J is just the empirical risk of the line, written as a function of its two weights. Training means finding the pair (w_1, w_0) that makes J smallest.

The ½ is a convenience: it cancels the 2 that appears when we differentiate.

DATA SPACE VS WEIGHT SPACE

Two pictures of the same problem

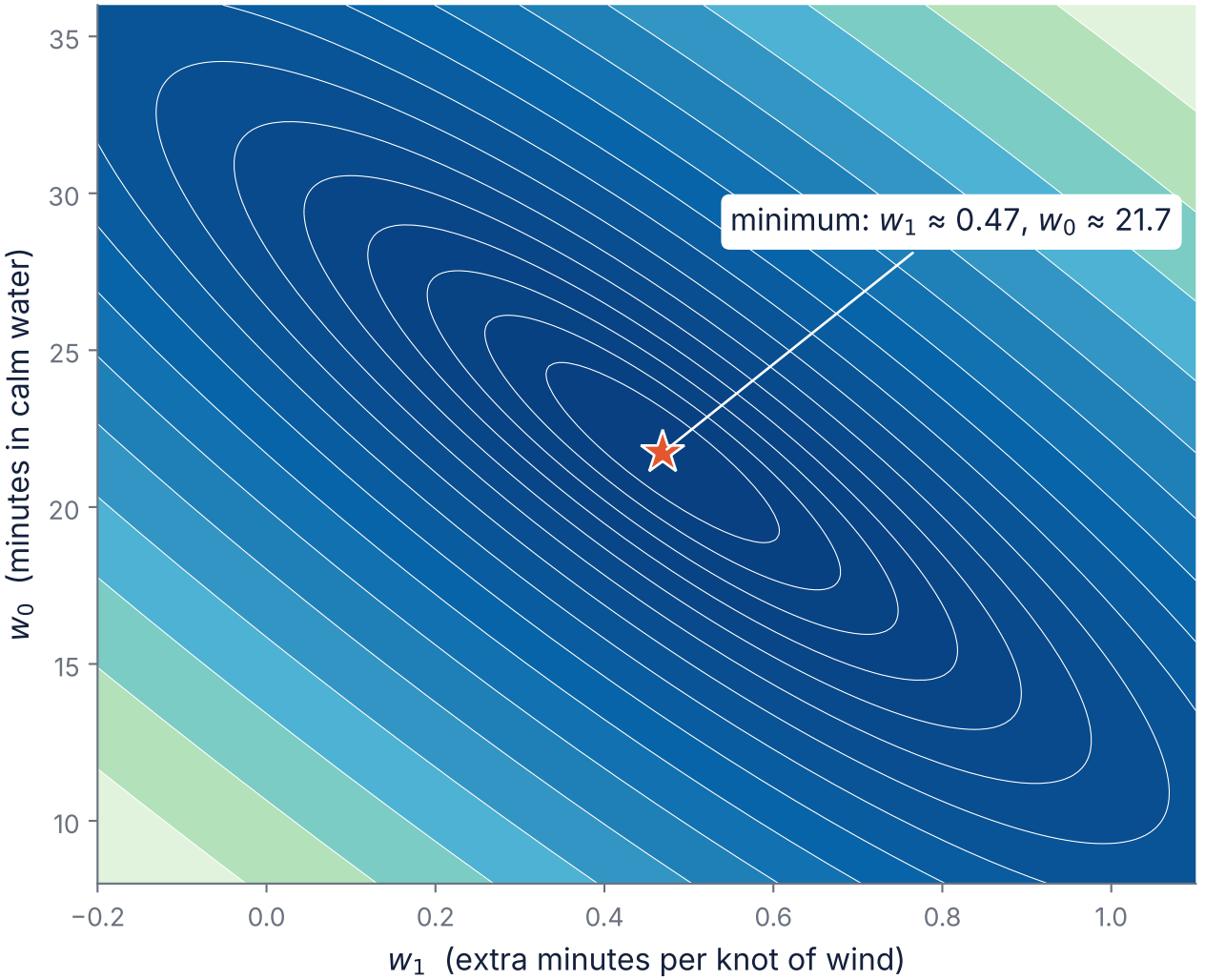




Same colour, same line. The best line on the left sits at the bottom of the bowl on the right.

WEIGHT SPACE

The cost surface over (w_1, w_0)



Each point is one possible line.

- Colour shows the cost J of that line
- Contours are ellipses around a single lowest point
- Training means finding the star
- The long diagonal valley: a steeper line with a lower start fits almost as well

EXAMPLE

The ferry log in Python

PYTHON

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(3151)
wind = rng.uniform(0, 35, 40)          # knots
minutes = 22 + 0.45 * wind + rng.normal(0, 2.5, 40)

log = pd.DataFrame({'wind_kn': wind,
                    'minutes': minutes})
log.head()
```

OUTPUT

	wind_kn	minutes
0	2.66	23.55
1	14.64	24.73
2	8.43	26.96
3	27.48	32.12
4	22.52	34.26

Synthetic data with a known answer: about 22 minutes in calm water plus 0.45 minutes per knot, plus noise.

EXAMPLE

Fitting the line in Python

PYTHON

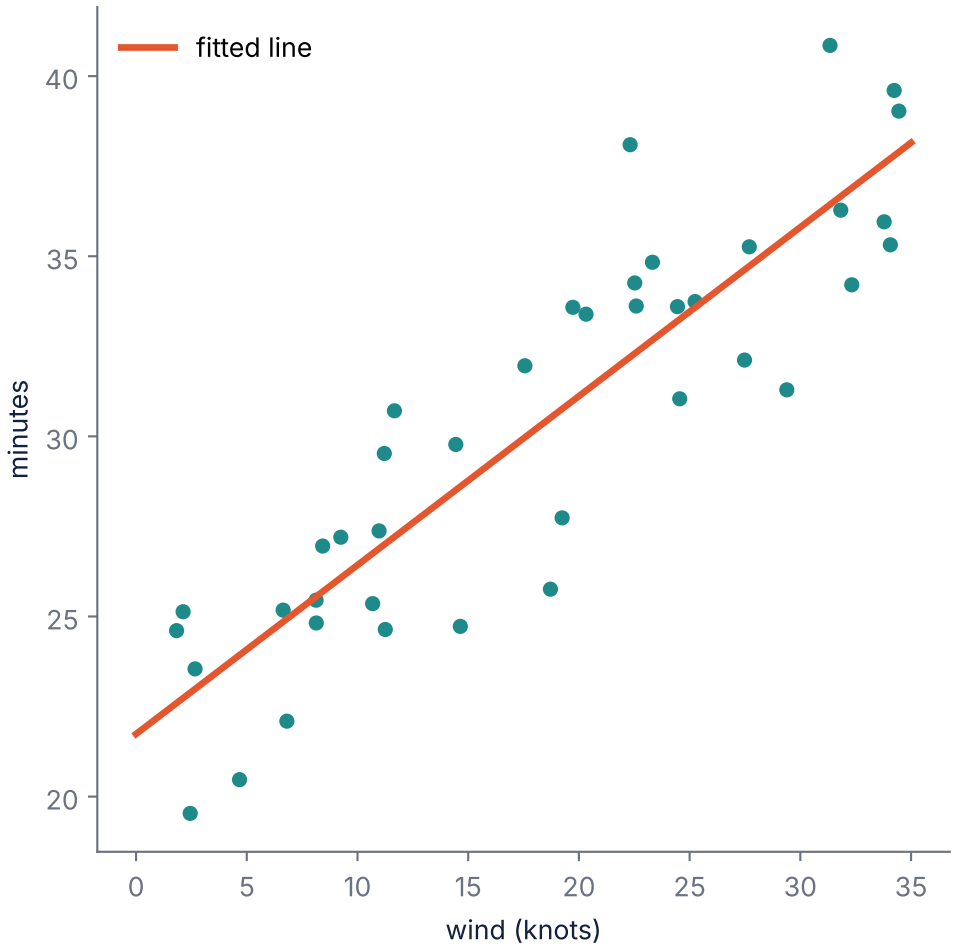
```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(log[['wind_kn']], log['minutes'])

model.coef_[0], model.intercept_
```

OUTPUT

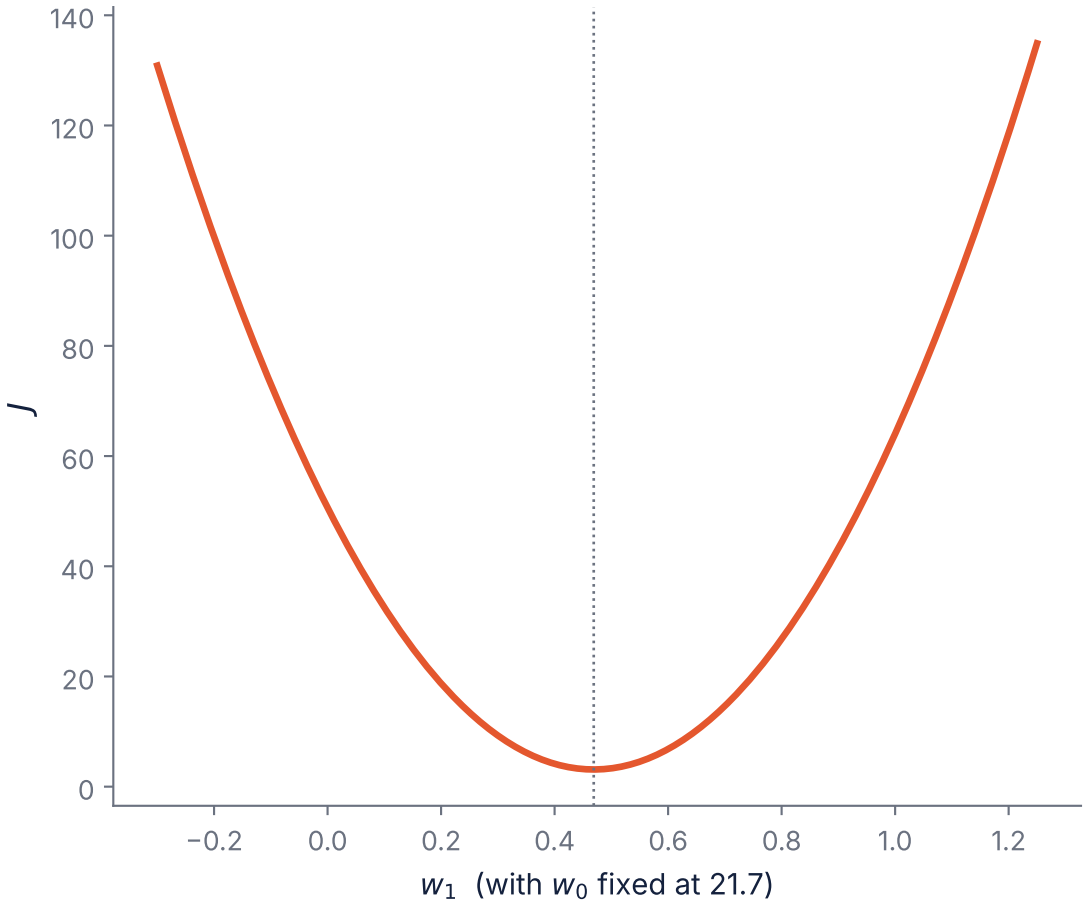
```
(0.469, 21.74)
```



About 21.7 min in calm water and 28 extra seconds per knot: close to the values used to make the data.

WEIGHT SPACE

Slicing the bowl



PYTHON

```
w1_grid = np.linspace(-0.3, 1.25, 200)
J = [np.mean((a*wind + w0 - minutes)**2) / 2
     for a in w1_grid]
```

Fix one weight and vary the other: every slice through J is a parabola with a single lowest point.

Stack all the slices together and you get the bowl from slide 12.

WEIGHT SPACE

Why one bowl? Quadratic and convex

Quadratic

Each residual $w_1 x^{(i)} + w_0 - t^{(i)}$ is linear in the weights. Squaring it gives terms no higher than w_1^2, $w_1 w_0$, w_0^2. Averaging keeps the degree at 2.

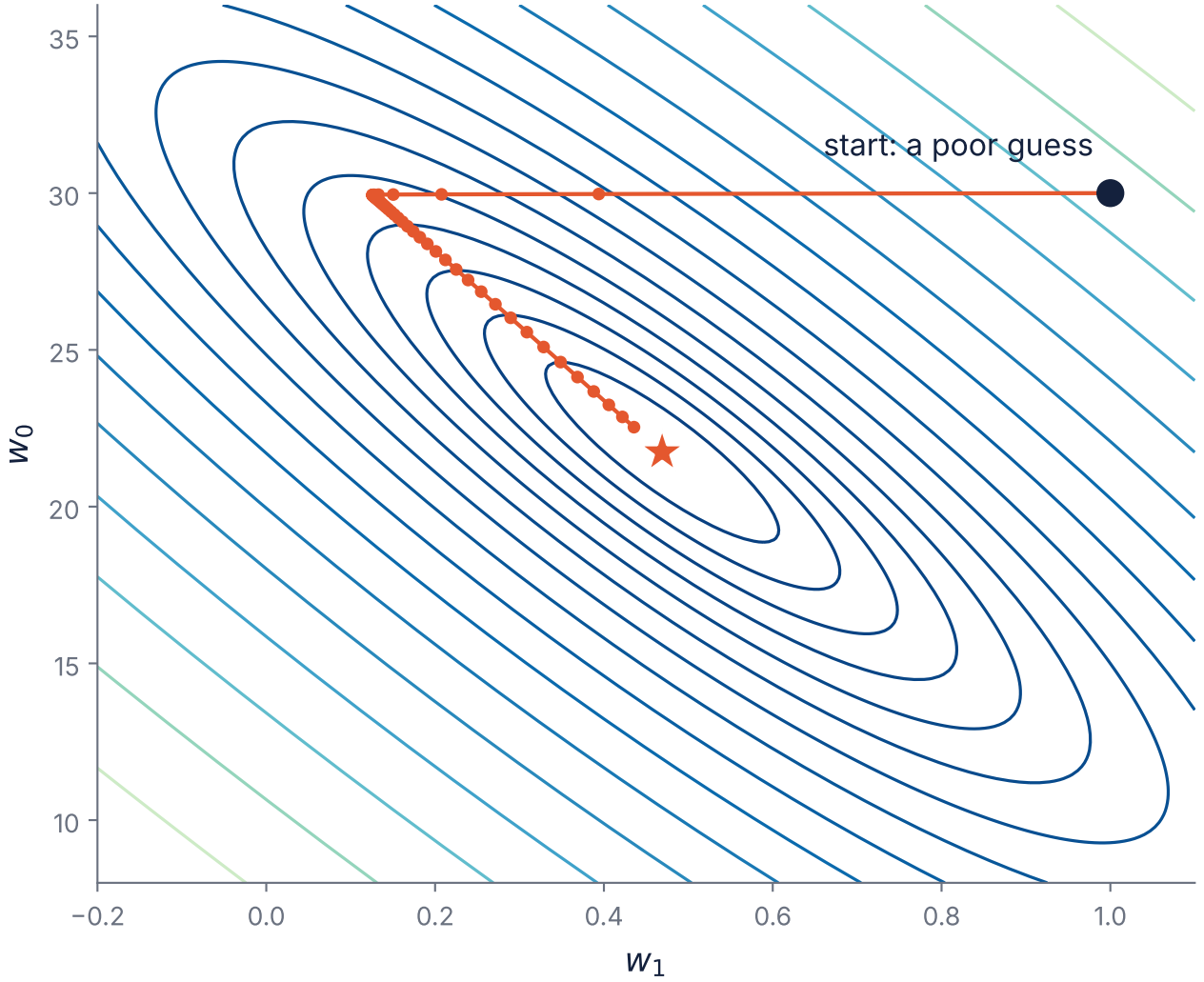
Convex

$z \mapsto z^2$ is convex. Plugging in a linear function of the weights keeps it convex, and an average of convex functions is convex too.

Consequence: there is no false bottom. Any point where the slope is zero is the global minimum, which is why gradient descent is dependable here.

OPTIMISATION

Gradient descent, drawn

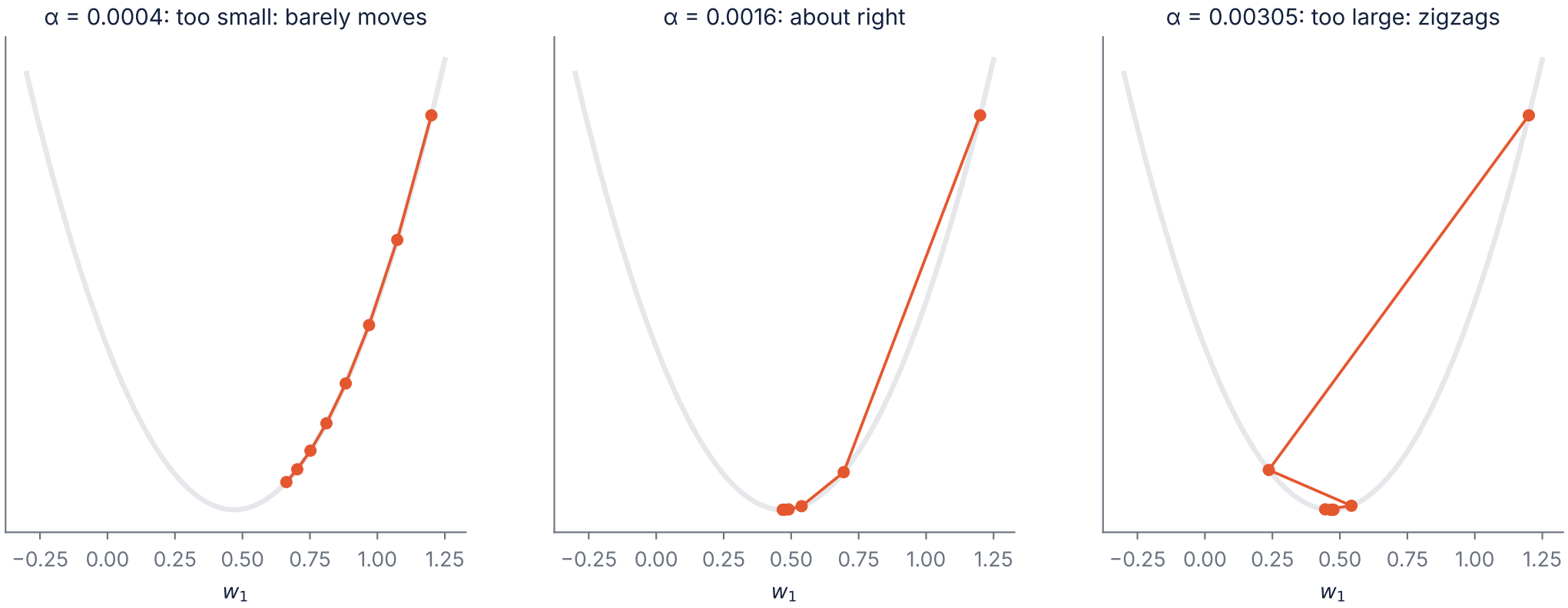


Walk downhill, one step at a time.

- Each step moves against ∇J
- First steps drop fast down the steep walls
- Then the path creeps along the narrow valley: rescaling the input would help
- The bowl is convex, so every downhill walk ends at the same minimum

OPTIMISATION

Step size matters



Eight steps each, same start, same slice of J as slide 15. The learning rate α is the only difference.

The update rule

Gradients

$$\frac{\partial J}{\partial w_1} = \frac{1}{n}\sum_i r^{(i)} x^{(i)} \qquad \frac{\partial J}{\partial w_0} = \frac{1}{n}\sum_i r^{(i)}$$

Update, repeated

$$w_1 \leftarrow w_1 - \alpha\, \partial J/\partial w_1 \qquad w_0 \leftarrow w_0 - \alpha\, \partial J/\partial w_0$$

OUTPUT

step	w1	w0	J
0	1.000	30.00	177.49
1	0.394	29.97	27.06
2	0.208	29.96	12.83
3	0.150	29.95	11.48
4	0.133	29.95	11.34

Starting from the poor guess on slide 17 (w_1 = 1, w_0 = 30) with α = 0.0016.

Many inputs: matrix form and the bias trick

Now each crossing has d inputs: wind (kn), wave height (m), passengers on board.

- Stack the n crossings as rows of X (n × d), targets in t
- Weights w (d numbers) plus a bias b
- Bias trick: add a column of ones to X and fold b into w

$$\mathbf{y} = X\mathbf{w}$$

$$J(\mathbf{w}) = \frac{1}{2n} \|X\mathbf{w} - \mathbf{t}\|^2$$

Still a convex quadratic in w, just in d + 1 dimensions instead of 2. Everything from the 1D case carries over.

CLOSED FORM

The normal equation

Set the gradient to zero

$$\nabla J = \frac{1}{n} X^\top (X\mathbf{w} - \mathbf{t}) = \mathbf{0}$$

$$\Rightarrow \quad X^\top X \mathbf{w} = X^\top \mathbf{t}$$

$$\Rightarrow \quad \mathbf{w}^* = (X^\top X)^{-1} X^\top \mathbf{t}$$

When to use it

Needs $X^\top X$ to be invertible (no perfectly redundant columns).

One solve, no learning rate, no iterations: great for a few hundred features.

With very many features the solve gets expensive; gradient methods scale better.

EXAMPLE

Closed form vs scikit-learn

PYTHON

```
Xb = np.c_[X, np.ones(len(X))]        # bias trick
w = np.linalg.solve(Xb.T @ Xb, Xb.T @ t)

sk = LinearRegression().fit(X, t)
print(w)
print(sk.coef_, sk.intercept_)
```

OUTPUT

```
true:        [0.4   3.0   0.02 ]  20.0
closed form: [0.391 2.974 0.0208] 19.89
sklearn:     [0.391 2.974 0.0208] 19.89
```

200 crossings, three inputs

Both routes land on the same weights, and both sit close to the values used to generate the data.

Waves add about 3 minutes per metre; each passenger adds about a second (boarding time).

Curves from a linear model: feature maps

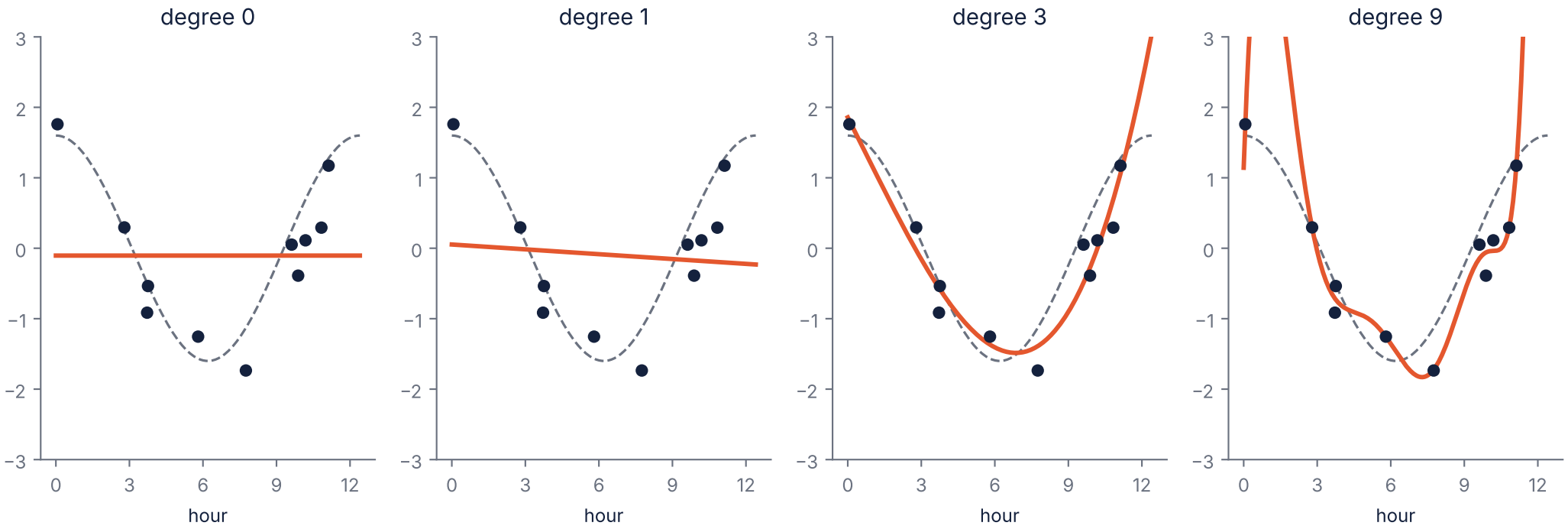
Tide height rises and falls over the day, so a straight line in time cannot follow it. Keep the model linear in the weights, but feed it new features:

$$\phi(h) = [\, h,\ h^2,\ h^3 \,] \qquad y = \mathbf{w}^\top \phi(h) + b$$

- Any fixed map φ works: powers, sines, bins, products of inputs
- The cost is still a convex quadratic in w, so all earlier tools still apply
- The curve in h can bend, even though the fit is linear regression

UNDER- AND OVERFITTING

Too simple, about right, too wiggly



Dark dots: 11 noisy tide readings. Dashed: the true tide curve. Red: polynomial fit of the given degree. Degree 9 threads through the noise instead of the tide.

EXAMPLE

Polynomial fits in scikit-learn

PYTHON

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline

for deg in [1, 3, 9]:
    m = make_pipeline(PolynomialFeatures(deg),
                      LinearRegression())
    m.fit(h_train, y_train)
    print(deg, mse(m, train), mse(m, val))
```

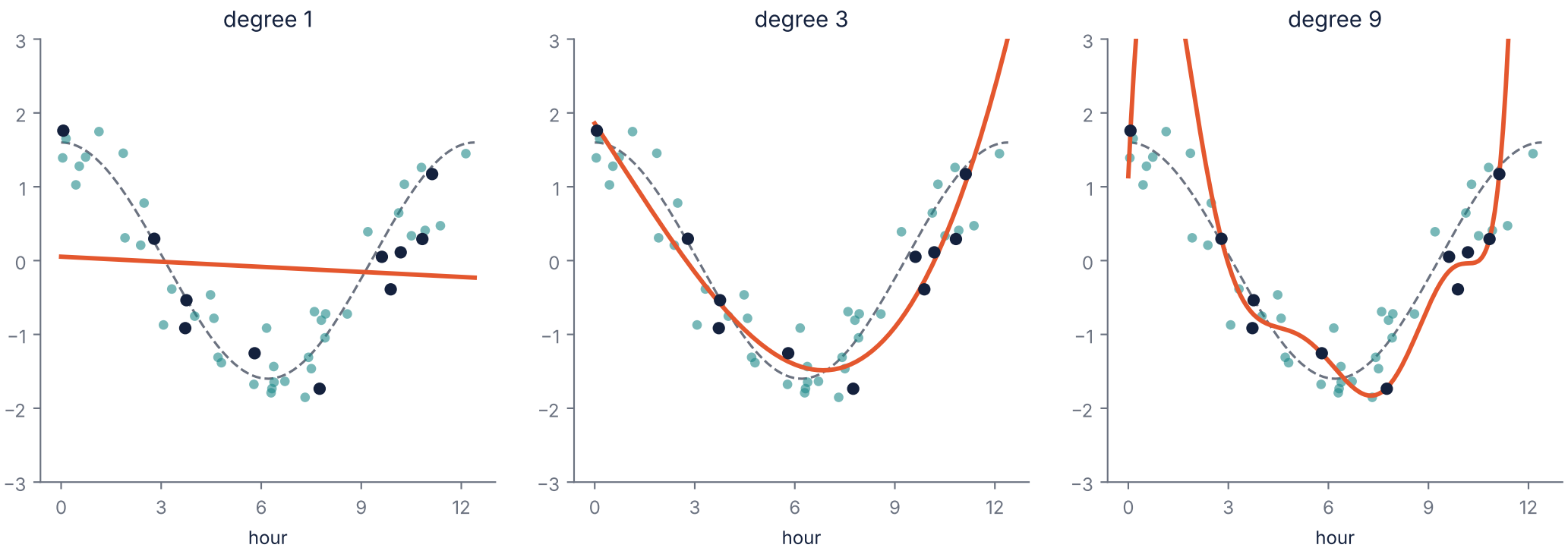
OUTPUT

```
deg   train MSE   val MSE
  1       0.939     1.309
  3       0.078     0.269
  9       0.023     9.282
```

Training error only goes down as the degree grows. Validation error is what tells the fits apart.

UNDER- AND OVERFITTING

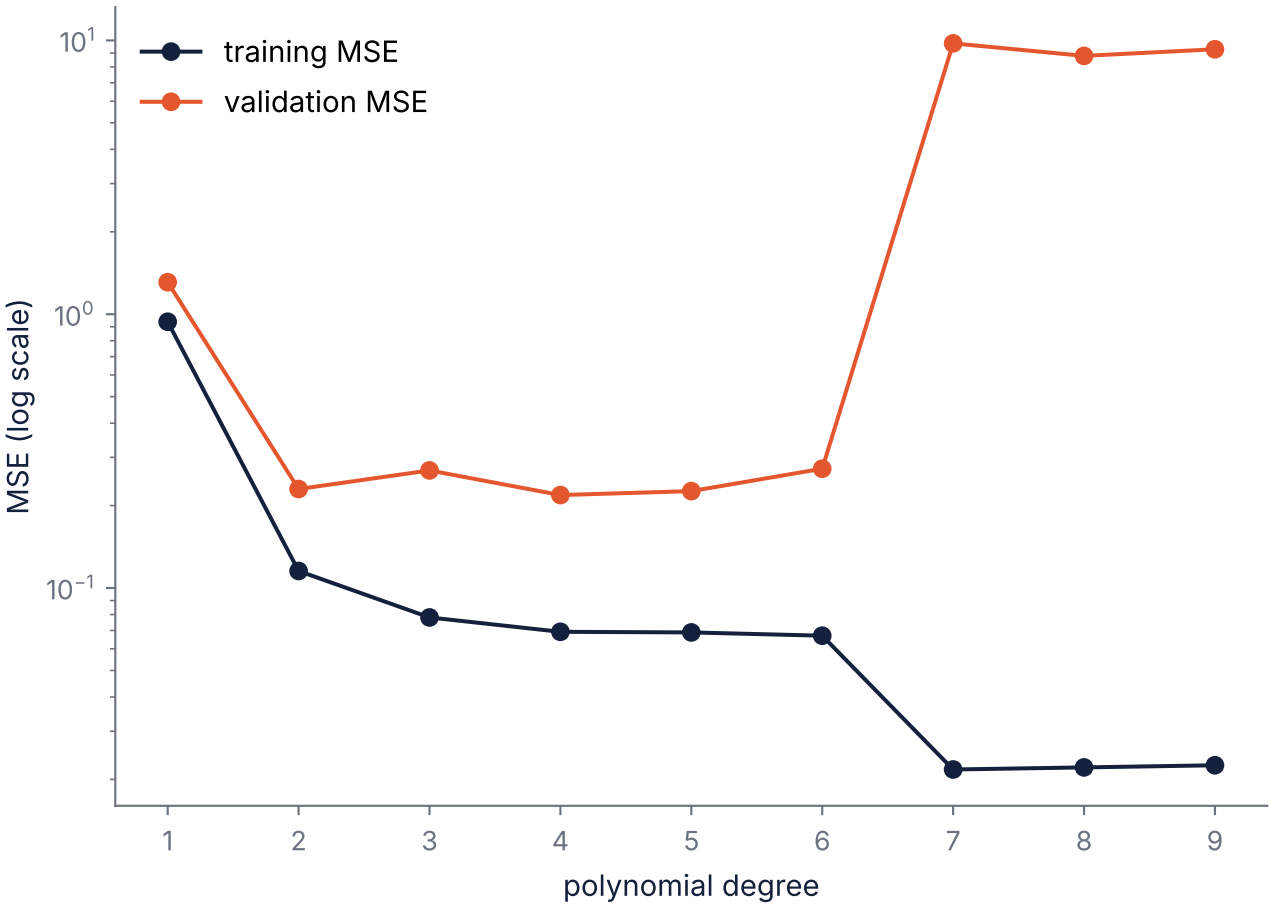
Seeing it against fresh data



Teal dots are 40 validation readings the fits never saw. Degree 1 misses the shape, degree 3 follows it, degree 9 swings away between the training points.

MODEL COMPLEXITY

Training vs validation error as the degree grows



- Training error keeps falling: more capacity always fits the log better
- Validation error falls, bottoms out, then climbs
- Pick the complexity at the bottom of the validation curve; the test set stays untouched

CLASSIFICATION

Losses for yes/no outcomes

New target: t ∈ {0, 1} (1 = crossing more than 10 minutes late). The model outputs p = its probability of a 1.

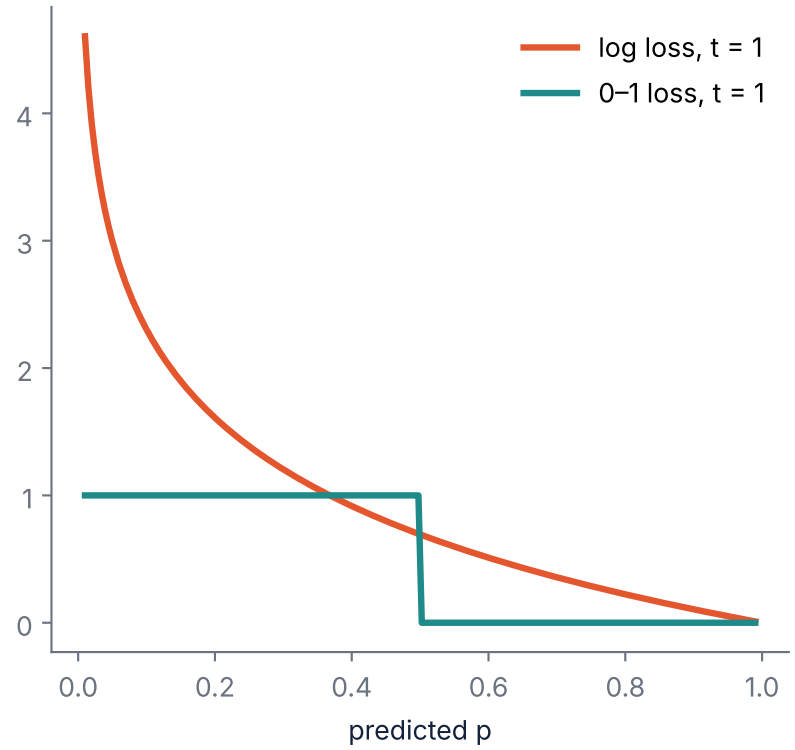
0–1 loss

1 if the rounded prediction is wrong, else 0. Easy to read, but flat almost everywhere, so gradients give no direction.

Log loss (cross-entropy)

$$\ell(p, t) = -\,[\, t \log p + (1 - t) \log(1 - p) \,]$$

Smooth, so gradient methods work. Confident mistakes cost a lot: p = 0.9 on an on-time crossing costs 2.30, p = 0.6 costs 0.92.



A TEMPTING SHORTCUT

Predicting a late crossing with a regression line?

Inputs: wind (kn) and wave height (m). Label: late = 1 if the crossing ran more than 10 minutes over schedule.

The shortcut

Treat the 0/1 label as a number, fit ordinary least squares, call it late when the prediction is ≥ 0.5.

Why it is shaky

Outputs are not confined to [0, 1], so they are not probabilities. Squared loss ignores that each outcome is a coin flip with some chance p.

It produces a classifier of sorts. The next slides show where it bends out of shape.

EXAMPLE

Trying the shortcut in Python

PYTHON

```
X = np.c_[wind, waves]                  # 160 crossings
reg = LinearRegression().fit(X, late)
pred = reg.predict(X)

print(pred.min(), pred.max())
print(((pred >= 0.5) == late).mean())
```

OUTPUT

```
-0.19 1.04
0.894
```

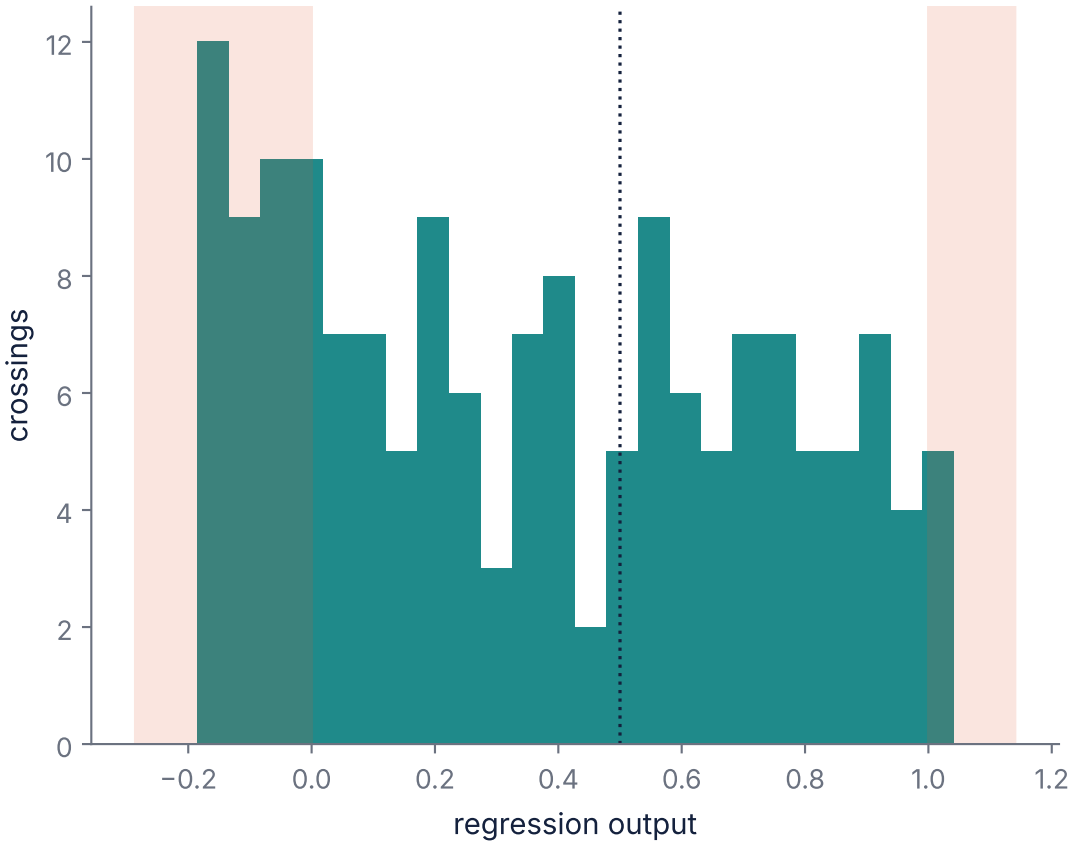
What came back

89% of crossings are labelled correctly, which sounds fine.

But 44 of the 160 'probabilities' fall below 0 or above 1.

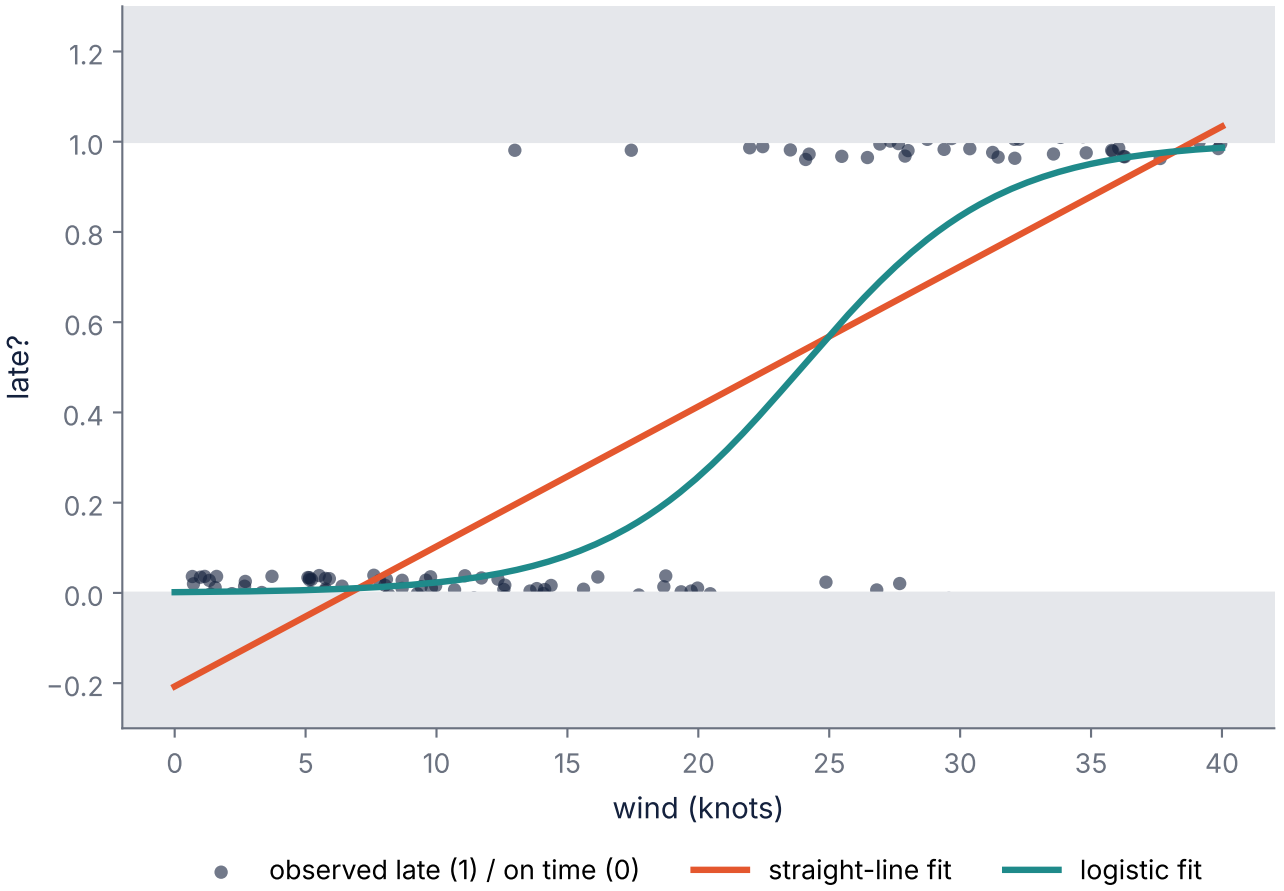
A TEMPTING SHORTCUT

Where the shortcut breaks



- Shaded areas: outputs that cannot be probabilities
- The 0.5 cut-off is arbitrary; nothing in the fit gives it meaning
- Very windy days push the line up for everyone and can move the cut-off for ordinary days

A line versus an S-curve



- The line runs past 0 and 1
- The S-curve stays inside [0, 1] and reads as a probability
- Fitting the S-curve with log loss is logistic regression: next module

BIG PICTURE

The same recipe at scale

Language models

Trained to predict the next token by minimising cross-entropy over a huge vocabulary.

Image classifiers

Cross-entropy again, now over thousands of classes.

Deep networks

Stacks of linear maps and nonlinearities, tuned by gradient steps on an average loss.

Data → model → loss → empirical risk → gradient steps. Once that chain makes sense for a ferry and a line, it makes sense for much larger systems.

QUICK QUESTIONS

Check yourself

1. A forecast misses by 3 minutes. How many times more does $(y - t)^2$ charge than $|y - t|$?

A. 3× B. the same C. 1.5× D. 9×

2. On the tide data, raising the polynomial degree from 3 to 9 most likely...

A. lowers both errors B. lowers training error, raises validation error
C. raises both errors D. changes nothing

3. Why is the normal equation less attractive with a million features?

A. J stops being convex B. it needs a learning rate
C. solving with X^TX becomes very expensive D. it can only fit one input

Answers: 1 — A (9 vs 3), 2 — B, 3 — C

LOOKING AHEAD

Where this goes next

Probability view

Squared loss and log loss come out of maximum likelihood once noise is modelled explicitly.

Logistic regression

The S-curve from slide 32, trained with log loss and gradient descent.

Evaluation

Validation curves, cross-validation, and what the bias–variance trade-off says about slide 27.

Regularisation

Penalties that tame high-degree fits, and optimisers that cope with big data.

REFERENCES

Further reading and credits

- Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Springer. Ch. 1 and 3
- Hastie, Tibshirani & Friedman (2009). The Elements of Statistical Learning, 2nd ed. Ch. 3
- scikit-learn user guide: Linear models; Polynomial features

About this lecture

Written by the LectureLoop team as the demo lecture for our app. All text, examples, figures and data are our own; the ferry, tide and lateness data are synthetic and generated by build_lecture.py. Released under CC BY 4.0.